```
#include<iostream>

#include<string>

 using namespace std;

struct Node { int data; Node* left; Node* right; };

Node* createNode(int value) { Node* newNode = new Node; newNode->data
= value; newNode->left = NULL; newNode->right = NULL; return
newNode; }

Node* insert(Node* root, int value) { if (root == NULL) { return
createNode(value); }

if (value < root->data) {
    root->left = insert(root->left, value);
} else {
    root->right = insert(root->right, value);
}

return root;


}

void inorder(Node* root) { if (root == NULL) { return; }

inorder(root->left);
cout << root->data << " ";
inorder(root->right);


}

void mirror(Node* root) { if (root == NULL) { return; }

Node* temp = root->left;
root->left = root->right;
root->right = temp;

mirror(root->left);
```

```
mirror(root->right);


}

void deleteTree(Node* root) { if (root == NULL) { return; }

deleteTree(root->left);
deleteTree(root->right);
delete root;


}

int main() { Node* root = NULL; int n, value;

cout << "Enter number of nodes: ";
cin >> n;

if (n < 0) {
    cout << "Invalid number of nodes.\n";
    return 0;
}

for (int i = 0; i < n; i++) {
    cout << "Enter node value: ";
    cin >> value;
    root = insert(root, value);
}

cout << "\nOriginal Tree (Inorder): ";
inorder(root);

mirror(root);

cout << "\nMirror Image (Inorder): ";
inorder(root);

cout << endl;

deleteTree(root);
```

```
return 0;

}
```